

Balthazar Wallet: Making Password Authentication Practical on Web3 via OPAQUE and Privacy-Preserving Smart Contracts

Tomas Krajci, Samuel Oleksak, Ivan Homoliak
 Brno University of Technology
 Faculty of Information Technology
 Brno, Czech Republic
 {ikrajci, ioleksak, homoliak}@fit.vut.cz

Abstract—Username & password is the most common authentication method in Web2 because of its high usability and efficient protection against brute-force attacks by applying rate limits on the server. In contrast, Web3 wallets cannot securely support password-derived keys. Passwords typically have low entropy, and because blockchain environments are public and impose no rate limits on brute-force attempts, attackers can repeatedly test guesses offline until the private key is recovered.

In this work, we present a novel password-based blockchain wallet that enables secure management of private keys (of any blockchain type) within a privacy-preserving smart contract platform (PPP). To store the keys, we adapt OPAQUE protocol to fit the decentralized environment of blockchains and leverage the properties of TEE within PPP. Our design consists of the client, relay, and smart contract deployed at PPP. To this end, we proposed the user enrollment protocol and private key retrieval protocol that require knowledge of the username with password and apply blockchain-enforced rate limits on the guesses.

Our implementation is based on the Oasis Sapphire confidential EVM as an instance of PPP. Our system implements OPAQUE’s Oblivious Pseudorandom Function (OPRF) inside a smart contract, allowing the contract to act as the protocol’s server while keeping the long-term OPRF key protected within the enclave. During authentication, the client performs a blinded OPRF interaction so that neither the password nor its derivatives are revealed to the relay, blockchain, or public. Experiments show that a single authentication attempt requires approx 500k and 300k of gas for 2048b and 1024b numbers within finite-field DLP, respectively; while one-time registration costs approx 270k of gas.

I. INTRODUCTION

Passwords have long been the dominant method for authenticating users to computer systems. Despite repeated attempts to replace them with alternative mechanisms, and despite their well-known usability and security limitations [1], they remain the primary authentication mechanism across modern digital services. In traditional client–server architectures, password-based authentication relies on a trusted server that stores password-derived information, known as *verifiers*. This model assumes that the server is both honest and capable of protecting the stored verifiers from disclosure.

In decentralized environments, however, these assumptions no longer hold. Public blockchains expose all contract code, state, and transaction inputs to every participant in the network. As a consequence, password-based authentication cannot be

deployed directly on a blockchain: storing password verifiers, executing secret-dependent computations, or even handling hashed passwords in plaintext would compromise user security. Similarly, password-derived wallets suffer from a low entropy of passwords and thus can be easily brute-forced with no rate limits. These limitations render password-authenticated systems (and wallets) fundamentally incompatible with transparent smart contract blockchain platforms.

OPAQUE & PPP Blockchains. Password-authenticated key exchange (PAKE) protocols, such as OPAQUE, provide stronger security guarantees by ensuring that the server never learns the password and that even compromised verifiers do not enable offline dictionary attacks [2]. However, OPAQUE relies on a long-term server-side secret key used during the Oblivious Pseudorandom Function (OPRF) evaluation, which must remain confidential. On public blockchains, where all data is openly accessible, secure server-side OPRF evaluation is not feasible. Nevertheless, there exists a category of blockchains built over TEE-compatible nodes, denoted as privacy-preserving smart contract platforms (PPP) a.k.a., confidential smart contract platforms [3]–[6]. With PPP, a blockchain platform could store a long-term secret of OPAQUE and act as a server, which motivates our work.

Our Work. We propose a practical password-protected wallet on top of a PPP by adapting OPAQUE protocol and its OPRF inside a privacy-preserving smart contract. The enclave protects the PPP platform’s long-term secret key (i.e., per user salt), while blinded OPRF evaluations ensure that clients can authenticate without ever revealing their passwords or password-derived values. To improve the user experience, we enriched our design with the relay service(s), enabling gasless onboarding and authentication via ephemeral wallets.

Our work demonstrates that password-based authentication and wallets, traditionally considered insecure to implement securely on public blockchains, become feasible when combined with privacy-preserving smart contract platforms. The evaluation of our prototype on a local Sapphire network shows that the system retains the strong security guarantees of OPAQUE while introducing minimal overhead, making decentralized password-protected wallets practical for real-world applications.

Organization. The remainder of this paper is organized as follows. Sec. II provides background on trusted execution environments, the transparency properties of Ethereum smart contracts, and the confidential execution model of the PPP. Sec. III formalizes the problem and reviews the limitations of classical password-based authentication and PAKE protocols in transparent blockchain environments. Sec. IV presents the proposed architecture for confidential OPAQUE-based authentication and describes the protocol workflows for registration and authentication. Sec. V evaluates the performance and security of the system on a local PPP test network. Sec. VI discusses related work, and Sec. VII concludes the paper.

II. BACKGROUND

In this section, we review preliminaries required to understand our approach. We will specifically focus on the utilization of the mentioned techniques for OPAQUE authentication.

A. Trusted Execution Environments (TEEs)

A trusted execution environment (TEE) is a secure area of a processor that provides confidentiality and integrity guarantees for code and data executed within it [7]. The code running inside a TEE is isolated from the host operating system, other applications, and even privileged system software. This isolation prevents external parties from inspecting or modifying the enclave's internal state. TEEs, such as Intel SGX, support remote attestation, allowing a remote party to verify that a specific piece of code is running inside a genuine enclave before transmitting sensitive information [8].

For privacy-preserving authentication, TEEs are particularly valuable because they allow the server-side components of password-authenticated key exchange (PAKE) protocols to run without exposing secret keys or intermediate computations. In the context of OPAQUE, the server must maintain a long-term (per user) private key for oblivious pseudorandom function (OPRF) evaluations. Protecting this key is essential: if leaked, an adversary could recover password-derived secrets or launch large-scale offline dictionary attacks. A TEE provides a secure execution boundary that ensures the key remains confidential even in the presence of a compromised operating system or a malicious infrastructure operator [9].

B. Ethereum & Transparency

Ethereum smart contracts execute within the Ethereum Virtual Machine (EVM), a deterministic state machine replicated across all full nodes [10]. Although this model ensures strong consistency and verifiability, it inherently lacks confidentiality. All contract storage, transaction calldata, execution traces, and emitted events are publicly visible. Even Solidity variables declared as `private` or `internal` do not provide secrecy. They merely restrict access at the source-code level, while the underlying storage remains fully accessible to any node.

This transparency makes it impossible to store password verifiers, execute OPRF computations, or maintain secret server-side keys on a standard EVM. Any attempt to deploy a password-based authentication protocol to a public blockchain

would immediately expose sensitive values to adversaries. As a result, conventional blockchain platforms are fundamentally incompatible with PAKE protocols that require server-side secrets and confidential computation [11].

C. Oasis Sapphire Confidential EVM

Oasis Sapphire extends the Ethereum execution model by integrating a trusted execution environment (TEE) directly into its smart-contract runtime [3]. Contracts deployed to Sapphire are executed inside secure enclaves, where contract storage is encrypted at rest, state transitions are encrypted in transit, and inputs and outputs can be selectively hidden from external observers. In addition to these confidentiality guarantees, Sapphire maintains full compatibility with existing Ethereum tools, libraries, and bytecode, enabling developers to use standard Solidity smart contracts while benefiting from enclave-backed security.

Sapphire's confidential EVM enables execution patterns that are not feasible on transparent blockchains. Sensitive variables, such as long-term OPRF keys, remain protected within the enclave and are used for computations on secret inputs. For example, evaluating $\beta = \alpha^{k_s} \bmod p$ in OPAQUE is performed without exposing intermediate results or key material. TEE-backed confidential contract platforms such as Ekiden [12], ShadowEth [13], and Fabric SGX extensions [14] also demonstrate the feasibility of combining TEEs with blockchain systems. However, Sapphire provides the first practical integration of TEEs with an Ethereum-compatible execution model, making it a suitable platform for implementing privacy-preserving authentication systems directly on-chain.

D. Notation

Throughout the paper, we use the following notation:

- $x \xleftarrow{R} S$ denotes sampling x uniformly at random from a set S ;
- $H(x)$ is a cryptographic hash function outputting a fixed-size value in $\{0, 1\}^n$;
- $H'(x)$ is a hash-to-integer or hash-to-group function used as input to the OPRF;
- $a^b \bmod p$ denotes modular exponentiation over a large prime modulus p ;
- $\text{AEnc}_k(m)$ and $\text{ADec}_k(c)$ denote authenticated encryption and decryption using AES-GCM with key k ;
- U is a user identifier (e.g., a username or a verified e-mail address);
- pwd is the user's password;
- k_s is the server-side OPRF private key provisioned to the enclave;
- $\beta = (H'(\text{pwd}))^{k_s} \bmod p$ is the unblinded OPRF output used to derive the envelope key;
- $k = H(\beta)$ is the symmetric key used to encrypt the credential envelope;
- (p_u, P_u) is the user's secret/public key pair stored inside the envelope;
- $c = \text{AEnc}_k(p_u, P_u)$ is the encrypted credential envelope;
- (x_u, X_u) is an ephemeral blockchain keypair used only to fund and sign the registration/authentication transactions;

- $\text{regs}[U]$ is a temporary registration record stored by the contract;
- $\text{users}[U] = (k_s, P_u, c)$ is the long-term confidential user metadata stored inside the PPP enclave.

These primitives correspond precisely to those used in our approach: A hash-based password pre-processing function, modular exponentiation for the OPRF evaluation, and AES-GCM for encrypting the user's credential envelope.

III. PROBLEM DEFINITION & DESIGN SPACE

A. Classical Password Authentication

The standard password-based authentication model consists of two steps:

- **Registration:** A user sends a username–password pair (U, pwd) over a secure channel. The server samples a random salt $s \xleftarrow{R} \{0, 1\}^n$ and stores $(s, H(s||\text{pwd}))$ under the identifier U .
- **Authentication:** The user sends (U, pwd') and the server verifies whether $H(s||\text{pwd}') = H(s||\text{pwd})$.

This model has a well-known limitation: the password is transmitted in plaintext at every login. Even with a secure channel, malware or compromised endpoints can expose passwords, while stolen verifier databases enable large-scale offline guessing attacks.

B. PAKE and OPAQUE OPRF

Password-authenticated key exchange (PAKE) protocols were introduced to address these shortcomings by allowing two parties to establish high-entropy keys from low-entropy passwords without revealing the password in plaintext [15]. OPAQUE [2] is a modern asymmetric PAKE that removes the need to reveal salts or password-derived values to the server, and prevents offline dictionary attacks even if the server's storage is compromised.

OPAQUE relies on an Oblivious Pseudorandom Function (OPRF). In our approach, the OPRF is instantiated via modular exponentiation over a large prime modulus p . For server key k_s and input m , the OPRF evaluation is:

$$f_{k_s}(m) = (H'(m))^{k_s} \bmod p. \quad (1)$$

To prevent the server from learning $H'(m)$, the client blinds the password input by computing:

$$\alpha = (H'(\text{pwd}))^r \bmod p, \quad (2)$$

for random $r \xleftarrow{R} \mathbb{Z}_{p-1}$, and sends α to the server. The server evaluates the OPRF:

$$\beta = \alpha^{k_s} \bmod p. \quad (3)$$

The client then unblinds the response using the modular inverse of r :

$$k = \beta^{1/r} \bmod p, \quad (4)$$

yielding the same OPRF output that the server would have computed on the unblinded password input.

C. Envelope Encryption

OPAQUE stores user-specific secret material in an encrypted *envelope* using a key derived from the OPRF output. We adopt the same approach: the client derives a symmetric key from k and encrypts its credential payload (typically a private key and its corresponding public key) using AES-GCM:

$$c = \text{AEnc}_k(\text{payload}). \quad (5)$$

During authentication, once the client recomputes the OPRF output, it decrypts the envelope:

$$\text{payload} = \text{ADec}_k(c). \quad (6)$$

Only a user with the correct password can reconstruct k and decrypt the stored envelope, ensuring strong confidentiality even if the server's encrypted data is exposed.

D. Security Assumptions

We assume the standard hardness of the discrete logarithm problem in the prime-order group used for modular exponentiation in the OPRF, as well as the security of the underlying hash functions, modeled as random oracles. AES-GCM is assumed to provide confidentiality and integrity under standard authenticated encryption security notions. For the system model, we assume the security of the PPP trusted execution environment, including enclave isolation and the correctness of remote attestation.

E. Goal

The goal of this work is to enable secure password-based authentication in decentralized blockchain environments while preserving the confidentiality guarantees required by modern password-authenticated key exchange protocols.

Specifically, we aim to design and implement an authentication system that allows users to authenticate with passwords without revealing the password, its hash, or any password-derived values to the blockchain, an intermediary relay service responsible for transaction forwarding and identifier verification, or smart contract operators. At the same time, the system must maintain the security guarantees of the OPAQUE PAKE protocol, including resistance to offline dictionary attacks and protection against server compromise.

To achieve this, the proposed system leverages confidential smart contract execution on a PPP, where trusted execution environments protect server-side secrets and enable the secure evaluation of the OPRF required by OPAQUE. The objective is therefore to demonstrate that password-authenticated identity systems can be realized securely in decentralized environments when combined with confidential computation.

F. Desired Properties

A decentralized password-authentication system deployed on a confidential smart-contract platform should satisfy the following properties:

- **Password confidentiality.** The user's password and any deterministic password-derived values must never be revealed

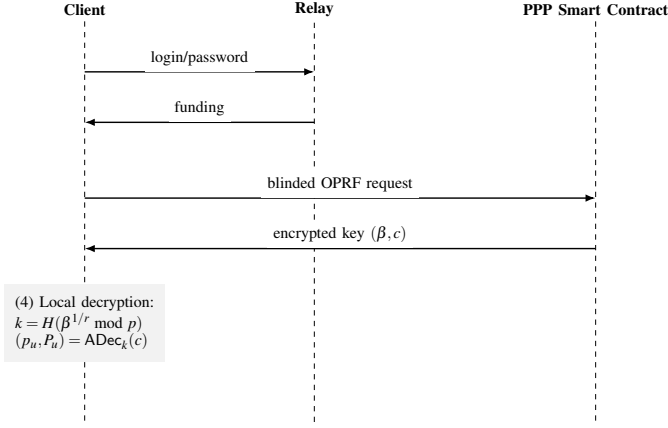


Fig. 1: System overview. The client authenticates using a login and password. The relay funds the ephemeral wallet and forwards requests to the PPP smart contract. The encrypted key is returned to the client, which decrypts it locally.

to the blockchain, the relay service, or the smart-contract operator.

- **Resistance to offline dictionary attacks.** An adversary obtaining the stored user metadata must not be able to perform offline password guessing. Each password attempt must require interaction with the live OPRF service.
- **Server compromise resilience.** Even if the contract’s encrypted storage is exposed, attackers should not be able to recover user credentials or derive password-related secrets.
- **Relay transparency.** The relay service may assist with identifier verification and transaction forwarding, but it must not learn any authentication secrets or gain the ability to impersonate users.
- **Blockchain confidentiality.** Sensitive values, including the OPRF key and credential envelopes, must remain protected from blockchain observers despite the transparent execution model of typical smart-contract platforms.
- **Practical deployability.** The system should remain compatible with existing Ethereum tooling and smart contract infrastructure while introducing minimal computational overhead.

IV. PROPOSED APPROACH

Our system consists of three cooperating components: a client application, a relay service, and a confidential smart contract deployed on a privacy-preserving smart contract platform (PPP). Together, these components implement a password-based authentication mechanism that follows the OPAQUE protocol while preserving the confidentiality of all password-derived values. We detail how the OPAQUE-based protocol is realized within the PPP confidential smart contract environment and explain the end-to-end workflows for user onboarding and subsequent login. See overview of our approach in Fig. 1.

A. Client

The client is responsible for pre-processing the password using hash-to-integer functions. It performs the OPRF evaluation locally during the registration phase. It generates a user secret

key pair that is encrypted and stored within the OPAQUE credential envelope. During authentication, the client constructs blinded inputs for the OPRF protocol. After receiving the server’s response, the client removes the blinding factor locally on the user’s device. Finally, the client decrypts the stored credential envelope to recover the protected key material. For each protocol execution, the client additionally generates an ephemeral blockchain key pair. This temporary wallet is used solely to sign a single transaction that covers the contract execution cost and is not involved in the authentication logic.

B. Relay

The relay is an off-chain service responsible for verifying user identifiers (e.g., email addresses) before allowing registration. It funds the user’s ephemeral wallet to enable gasless onboarding. It forwards both registration and authentication requests to the PPP smart contract. The relay never observes the user’s password, the OPRF input, or any other secret material. With respect to confidentiality, the relay is considered *untrusted*. Even a malicious relay cannot learn any password-derived values because the blinded OPRF inputs reveal no information about the underlying password.

C. PPP Smart Contract

The smart contract is deployed on a privacy-preserving smart contract platform (PPP), where all execution occurs inside a TEE enclave. The contract receives user metadata during the registration phase. It stores the server’s long-term OPRF key k_s within the protected enclave state. It also stores the encrypted credential envelope c associated with the user. During authentication, the contract evaluates the OPRF using the enclave-protected key. The resulting blinded OPRF response is then returned to the client. Because the enclave protects both contract execution and storage, the long-term OPRF key k_s , stored metadata, and all intermediate computations remain confidential even from node operators.

D. Registration Workflow

The goal of the registration phase is to provision the long-term PPP platform’s per-user OPRF key k_s and the user’s encrypted credential envelope c into the PPP contract while ensuring that no password-derived values are exposed to the relay, the blockchain, or the contract operator. Fig. 2 summarizes the interaction.

1) *Client-side Preparation:* The user locally derives the OPRF output $\beta = (H'(pwd))^{k_s} \bmod p$ and computes the envelope-encryption key $k = H(\beta)$. Using this key, the client encrypts its credential payload (p_u, P_u) into an authenticated encryption envelope $c = \text{AEnc}_k(p_u, P_u)$. The user also generates an ephemeral blockchain keypair (x_u, X_u) , used only to sign and pay for the registration transactions. At no point does the client reveal pwd , β , k , p_u , or k_s to any external party.

2) *Relay-assisted Identifier Validation:* The user sends only (U, X_u) to the relay. The relay verifies that the identifier U belongs to the user (e.g., through e-mail validation) and funds the ephemeral wallet X_u . Therefore, registration can proceed without requiring the user to pay for gas. The relay does not receive any password-derived information.

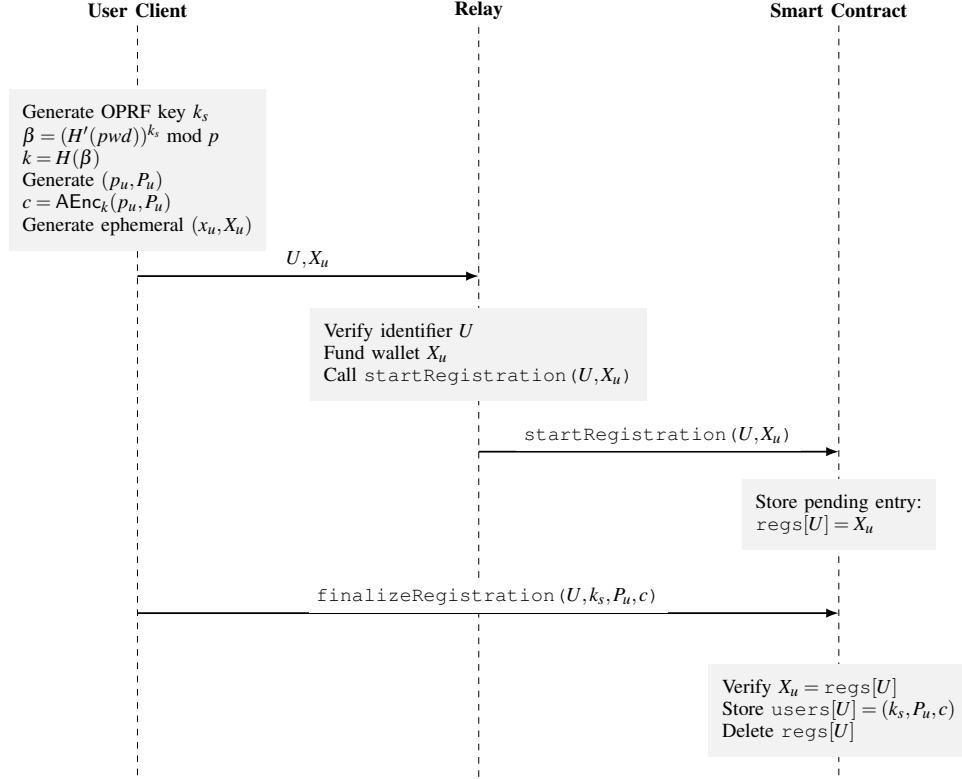


Fig. 2: Registration protocol. The user derives the envelope encryption key locally using the OPAQUE OPRF, constructs the encrypted credential envelope c , and provisions the server-side OPRF key k_s and public key P_u to the PPP contract. The relay only verifies identifiers and funds the ephemeral wallet used to pay for on-chain execution. All sensitive values are stored confidentially inside the contract’s TEE-protected state.

3) *Contract-side Initialization:* The relay invokes $\text{startRegistration}(U, X_u)$, causing the contract to store a pending entry $\text{regs}[U] = X_u$. This step prevents identifier squatting by ensuring that only the entity controlling X_u may finalize the registration. Because the contract executes inside a PPP enclave, this state is stored confidentially.

4) *Finalization:* After the relay completes its checks, the user submits $\text{finalizeRegistration}(U, k_s, P_u, c)$ directly to the contract. The contract verifies that the sender’s ephemeral public key matches the previously stored $\text{regs}[U]$, then stores the confidential record

$$\text{users}[U] = (k_s, P_u, c)$$

inside the encrypted enclave storage and deletes the pending entry. Because all OPRF material and envelope data reside exclusively inside the TEE, no external observer can recover password-derived secrets.

5) *Security Properties:* This two-step design prevents identifier hijacking and avoids exposing password-derived values to the relay or the blockchain, and ensures that long-term secrets are always sealed in the enclave. The relay is not trusted with authentication secrets; its only role is to validate user identifiers and to provide gas for onboarding. All subsequent authentication steps operate exclusively using the protected state stored within the PPP contract.

E. Authentication Flow

The authentication phase allows a registered user to prove knowledge of the correct password without ever sending the password or any deterministic password-derived value to the relay or the blockchain. Instead, the client performs a blinded OPRF query to the PPP contract and uses the resulting OPRF output to decrypt the credential envelope stored during registration.

1) *Client-side Blinding and Request:* Given a username U and password pwd , the client chooses a fresh random blinding exponent $r \xleftarrow{R} \mathbb{Z}_{p-1}$ and computes

$$\alpha = (H'(pwd))^r \bmod p.$$

The value α is a blinded representation of the password input to the OPRF. The client also prepares or reuses an ephemeral blockchain keypair (x_u, X_u) that will receive the response. It then sends (U, α, X_u) to the PPP contract (optionally via the same relay, which may again sponsor gas but does not see the password, the envelope, or the OPRF key). The result of this step needs to be confirmed in the blockchain to avoid offline brute-forcing by the platform’s operator. Hence, each password guess attempt costs the attacker/user transaction fees, while there is a rate limit on the number of password guesses per unit of time. Moreover, each password guess is visible on the blockchain, and therefore, the user is informed that somebody is brute-forcing her account.

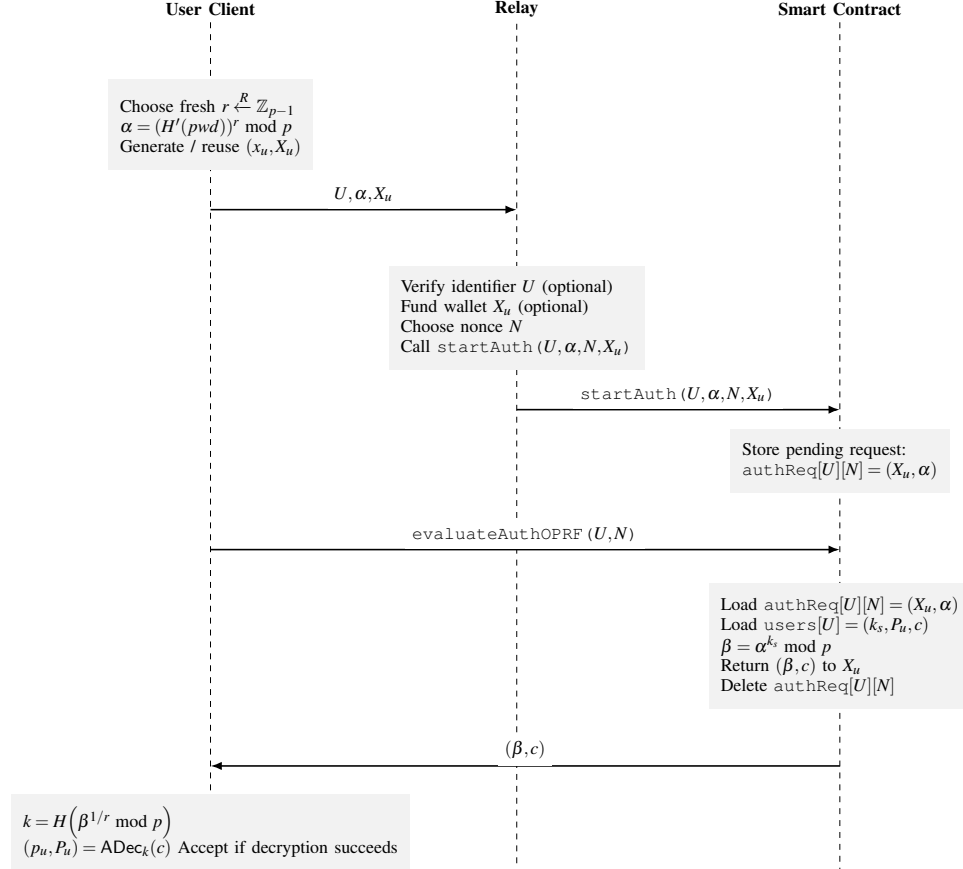


Fig. 3: Authentication protocol. The user blinds the password input into α and sends (U, α, X_u) to the PPP contract (optionally via a relay). The relay submits startAuth to create a pending request that serves as a means to protect against replay attacks on TEE. The user then calls $\text{evaluateAuthOPRF}(U, N)$, causing the contract to evaluate the OPRF using the enclave-protected key k_s and return (β, c) . After unblinding β and recomputing k , the user decrypts the credential envelope. Successful decryption proves knowledge of the correct password without revealing it or any of its derivatives.

2) *Contract-side OPRF Evaluation:* Upon receiving (U, α, X_u) , the contract looks up the corresponding user record

$$\text{users}[U] = (k_s, P_u, c)$$

stored inside the enclave during registration. The contract then evaluates the OPRF on the blinded input using the secret key k_s :

$$\beta = \alpha^{k_s} \bmod p.$$

Because the OPRF key k_s resides only inside the TEE, no node or operator learns it, and the computation proceeds confidentially. The contract returns β together with the stored envelope c to the caller over PPP's encrypted channel.

3) *Client-side Unblinding and Envelope Decryption:* On receiving (β, c) , the client removes the blinding factor by computing

$$k = H(\beta^{1/r} \bmod p).$$

which yields the same OPRF-derived key k that was used during registration. Using this key, the client decrypts the credential envelope:

$$(p_u, P_u) = \text{ADec}_k(c).$$

If decryption fails or the authenticated encryption check fails, the client rejects the authentication attempt. Otherwise, the

user has demonstrated knowledge of the correct password and recovered its long-term key material (p_u, P_u) for subsequent use (e.g., to authenticate to higher-level applications or derive session keys).

4) *Security Properties:* The authentication protocol enforces that every password guess must be processed online by the PPP contract. An adversary controlling the relay or observing the blockchain sees only blinded inputs α and never learns k_s , $\beta^{1/r}$, k , or the cleartext contents of the envelope. Even if the enclave's persistent storage were exposed, the attacker would obtain only (k_s, P_u, c) , which does not enable offline dictionary attacks without access to the online OPRF oracle. Thus, the system inherits OPAQUE's resistance to server compromise and provides password-based login with strong confidentiality guarantees in the smart-contract setting.

F. Security Analysis and Discussion

We analyze the security of the proposed system against the standard threat model for PAKE protocols and confidential computing platforms. The objective is to determine whether the confidential EVM provided by PPP preserves OPAQUE's security properties while introducing no new attack vectors.

We assume the standard security assumptions about utilized cryptographic primitives as well as a secure PPP platform.¹

- **Password secrecy:** The password, its hash-to-group image, and the unblinded OPRF output never leave the client.
- **Relay compromise:** The relay observes only blinded inputs $\alpha = H'(pwd)^r$, which leak no information about the password.
- **Blockchain transparency:** All contract state and execution traces are encrypted; events emitted from the enclave do not reveal secret data.
- **Replay protection:** Nonces, pending request records, and state cleanup ensure that every OPRF evaluation is bound to a unique authentication attempt whose initiation is captured by the blockchain’s state – this renders each password guessing attempt to cost transaction fees and be visible.
- **Resistance to offline attacks:** The OPAQUE design ensures that no password-derived verifier is ever stored in the clear, preserving offline guessing resistance.

Overall, the design inherits OPAQUE’s strong security guarantees while integrating seamlessly with the confidential EVM execution model.

V. IMPLEMENTATION AND EVALUATION

To assess the feasibility and practicality of running a password-authenticated key exchange (PAKE) protocol on a confidential smart contract platform, we evaluated our implementation along three key axes: performance, security, and deployability. All experiments were conducted on the Sapphire local network environment, which replicates the confidential EVM execution environment with the same SGX-backed semantics and encrypted state transitions.

A. Experimental Setup

We deployed the smart contract and executed the registration and authentication workflows using the TypeScript client and relay service described in Sec. IV. The evaluation environment consisted of the following:

- Sapphire Localnet container (Oasis Protocol Foundation, 2025).
- Hardhat EVM tooling for deployment and scripting.
- Node.js TypeScript client for OPRF, AES-GCM, and unblinding operations.
- Ephemeral wallets for gasless user onboarding.
- EVM precompile 0x05 for modular exponentiation.

B. Performance of On-chain OPRF Evaluation

The most computationally expensive on-chain operation is the OPRF computation:

$$\beta = \alpha^{k_s} \bmod p,$$

¹Note that if PPP platform were to be compromised by a hardware attack on TEE, the attacker would obtain (k_s, P_u, c) , which enables offline guessing without access to the live OPRF oracle.

Tab. I: Gas Cost and Latency of Core Operations

Operation	Gas Cost (avg)		Latency	Notes
	2048-bit	1024-bit		
startRegistration	~111,000	~111,000	<4023 ms	Store eph. X_u
finalizeRegistration	~164,000	~164,000	<4102 ms	Write k_s, P_u, c
startAuth	~338,000	~213,000	<4059 ms	Store α, N
evaluateAuthOPRF	~158,000	~87,000	<4115 ms	OPRF modexp + return

executed inside the Sapphire enclave using the standardized modular exponentiation precompile. Tab. I summarizes the average gas consumption and latency for two variants of integer sizes for finite field DLP – i.e., 2048b and 1048b.²

These results confirm that the OPRF computation, which is the core of OPAQUE, can be executed with minimal overhead within a TEE-protected EVM environment. The total on-chain cost of a complete authentication flow, comprising startAuth and evaluateAuthOPRF, amounts to approximately 496,000 gas, with each operation completing in under 4,200 ms. These figures demonstrate that password-authenticated key exchange can be realized on a confidential smart contract platform with latencies acceptable for real-world login flows.

C. Feasibility and Practicality

Our prototype demonstrates that running modern password-protected wallets on-chain is now viable. By executing sensitive computations inside a TEE, the fundamental confidentiality limitations of classical blockchains are eliminated. The use of EVM precompiles ensures that the OPRF remains efficient even with 2048-bit safe primes, while ephemeral wallets and relay-assisted funding enable gasless onboarding without exposing any secrets to the relay. Furthermore, the design relies exclusively on standard Solidity and TypeScript tooling, confirming that Oasis Sapphire provides a realistic and practical environment for deploying privacy-preserving authentication systems at scale.

VI. RELATED WORK

This work lies at the intersection of confidential smart contract platforms, password authentication, and decentralized identity management. We review the individual fields below and position our contribution within them.

A. Smart Contract Platforms

Multiple projects have explored the idea of using TEE to back the confidentiality of smart contracts. Ekiden [12], ShadowEth [13] and the extension of Hyperledger called Fabric Private Chaincode (FPC) each run contract code inside a secure enclave while only recording encrypted state on-chain.

Secret Network [4] was the first platform to deploy privacy-preserving smart contracts on a public mainnet, combining Intel SGX with a Cosmos SDK-based chain to provide encrypted

²Note that finite field DLP was already computed for 795-bit long safe prime [16]; the computation took 3100 core-years, using Intel Xeon Gold 6130 (2.1GHz) CPUs.

inputs, outputs, and contract state by default. However, Jean-Louis et al. [17] audited four TEE-based platforms – Secret Network, Oasis, Phala, and Obscuro – and found practical vulnerabilities (access-pattern leakage, state-consistency violations) that do not require breaking the TEE itself, showing that enclave confidentiality alone is not sufficient without careful contract and platform design.

Oasis Sapphire [3] extends the Ethereum execution model with an SGX-backed confidential EVM that maintains full EVM compatibility, allowing developers to use standard Solidity tooling while benefiting from encrypted contract storage and confidential state transitions. Sapphire addresses some of the state-consistency concerns raised by Jean-Louis et al. through built-in rollback protections. Among the few research-grade systems built on Sapphire, Austgen et al. [18] present Liquefaction, a key-encumbrance wallet that uses Sapphire’s confidential execution to attach rich, multi-user access-control policies to private keys, enabling assets to be privately rented, shared, or pooled. The Oasis ecosystem also includes production applications for confidential voting, sealed-bid auctions, and privacy-preserving random number generation, though these remain largely in the industry domain. Our system leverages Sapphire to host the OPRF server logic, demonstrating that the confidential EVM is expressive enough to support cryptographic authentication protocols directly on-chain.

B. PAKE

Password-authenticated key exchange (PAKE) protocols date back to the Encrypted Key Exchange (EKE) by Bellare and Merritt [15]. Subsequent asymmetric variants ensure the server only stores a password-derived verifier and never learns the plaintext password. OPAQUE, proposed by Jarecki, Krawczyk, and Xu [2], is a modern asymmetric PAKE that uses an Oblivious Pseudorandom Function (OPRF) to keep the salt completely secret from the client and the password completely hidden from the server. Unlike earlier aPAKE protocols such as SRP [19], AugPAKE [20], or SPAKE2+ [21], OPAQUE provides formal security proofs in the Universal Composability model and is resistant to pre-computation attacks upon server compromise. OPAQUE was selected by the IRTF Crypto Forum Research Group (CFRG) in their 2019 aPAKE selection process and has since been standardized in July 2025 as RFC 9807 [22]. Our implementation follows the OPAQUE construction but adapts it for the on-chain setting by placing the OPRF server-side evaluation inside a PPP smart contract enclave.

C. Password-Authenticated DIDs

The work most closely related to ours is the Password-authenticated Decentralized Identities (PDID) framework proposed by Szalachowski et al. [23]. PDID was the first system to combine a modified OPAQUE protocol with confidential smart contracts to enable password-based authentication on the blockchain. The implementation of PDID leverages Hyperledger Fabric Private Chaincode (FPC), utilizing Intel SGX to execute the OPRF evaluation within an enclave. The PDID implementation has achieved human-understandable, globally

unique, password-authenticated identities in a decentralized setting. However, its identity management relies on selective disclosure and zero-knowledge techniques, while PAKE-based authentication serves only as the login mechanism.

In contrast, our approach leverages OPAQUE and TEE-backed confidential smart-contract execution to enable password-based authentication directly on-chain, complementing PDID by providing a secure server-side OPRF that preserves password confidentiality even in decentralized environments. Our work differs from PDID in several important respects. First, we target a public, EVM-compatible confidential blockchain (Oasis Sapphire) rather than the permissioned Hyperledger Fabric platform, enabling fully permissionless deployment and interoperability with the Ethereum ecosystem. Second, our architecture introduces a relay service that handles identifier verification and gasless onboarding via ephemeral wallets, lowering the barrier to entry for end users. Third, whereas PDID instantiates the modified OPAQUE protocol using elliptic-curve primitives in C/C++ within the FPC framework, our implementation uses the EVM modular exponentiation precompile (0x05) to perform the OPRF directly in Solidity.

The broader decentralized identity landscape relies predominantly on public-key cryptography rather than passwords. The W3C Decentralized Identifiers (DIDs) specification [24] and Verifiable Credentials (VCs) [25] enable selective disclosure of identity attributes through cryptographic proofs but require users to manage key material directly – a well-known usability challenge [1]. Zero-knowledge proof (ZKP) techniques offer an alternative path: systems such as zkLogin [26] allow users to authenticate using existing OAuth credentials combined with zk-SNARKs to prove identity on-chain without revealing the underlying data. However, ZKP-based authentication typically requires computationally expensive proof generation on the client side and relies on external identity providers. Our TEE-based approach avoids these proof-generation costs, does not depend on third-party identity providers, and directly supports the familiar password-based login model.

D. Cryptocurrency Wallet Classification

Within the classification of cryptocurrency wallets by the number and type of authentication factors created by Homoliak and Perešini [27], our system is classified as a 1+1 wallet – the user’s password (knowledge factor) and the enclave-held OPRF key (server-side factor) are both required to reconstruct credentials. This means that neither factor alone is sufficient – the password cannot reconstruct the credential envelope without interacting with the on-chain OPRF oracle, and conversely, the enclave cannot derive user credentials without the correct password input. To the best of our knowledge, this password-plus-enclave two-factor structure has not been explored in prior wallet designs.

VII. CONCLUSIONS

In this work, we presented Balthazar, a novel password-based blockchain wallet that enables secure storage and retrieval of private keys using a privacy-preserving smart

contract platform. By adapting the OPAQUE protocol to the decentralized blockchain environment and leveraging the confidentiality guarantees of TEE-backed smart contract execution, we demonstrated that password-based authentication, long considered incompatible with public blockchains, can be realized securely and practically. Our proposed architecture consists of three cooperating components: a client, a relay service, and a confidential smart contract. Together, they enforce that no password-derived values are ever exposed to the blockchain, the relay, or the contract operator, while applying rate limits on authentication attempts to resist brute-force attacks. A formal security analysis confirms that the system inherits OPAQUE’s strong guarantees, including resistance to offline dictionary attacks and server compromise. We evaluated our approach on Oasis Sapphire as a concrete instance of a PPP, where experiments on a local testnet confirmed that the system introduces minimal computational overhead, with a complete authentication requiring approximately 500k and 300k of gas for 2048b and 1024b long integers. These results highlight that confidential smart contract platforms open the door to a new usability class of decentralized applications that were previously impossible with smart contract wallets of transparent blockchains.

REFERENCES

- [1] Joseph Bonneau, Cormac Herley, Paul C Van Oorschot, and Frank Stajano. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012.
- [2] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. Opaque: an asymmetric pake protocol secure against pre-computation attacks. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2018.
- [3] Oasis Protocol Foundation. Oasis sapphire: Confidential evm. <https://docs.oasis.io/dapp/sapphire/>, 2022. Accessed: 2025-01-01.
- [4] Secret Network. Secret network documentation, 2024. Accessed: 2025-05-01.
- [5] Hang Yin, Shunfan Zhou, and Jun Jiang. Phala network: A confidential smart contract network based on polkadot. Technical report, Phala Network, 2019.
- [6] Integritee AG. Integritee network documentation, 2024. Accessed: 2025-01-01.
- [7] Victor Costan and Srinivas Devadas. Intel sgx explained. *IACR Cryptology ePrint Archive*, 2016:86, 2016.
- [8] Ittai Anati, Shay Gueron, Simon Johnson, and Vincent Scarlata. Innovative technology for cpu based attestation and sealing. In *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy*, 2013.
- [9] Ferdinand Brasser, Ahmad-Reza Sadeghi, et al. Constant-time end-to-end secure computation and storage with sgx. *arXiv preprint arXiv:1906.02369*, 2019.
- [10] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Technical report, Ethereum Project Yellow Paper, 2014.
- [11] Joseph Bonneau, Cormac Herley, Paul C Van Oorschot, and Frank Stajano. Sok: Passwords and the evolution of imperfect authentication. *Communications of the ACM*, 58(7):78–87, 2015.
- [12] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah Johnson, Ari Juels, Andrew Miller, and Dawn Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019.
- [13] Rui Yuan, Yu-Bin Xia, Hai-Bo Chen, Bin-Yu Zang, and Jan Xie. Shadoweth: Private smart contract on public blockchain. *Journal of Computer Science and Technology*, 33(3), 2018.
- [14] Marcus Brandenburger, Christian Cachin, Rüdiger Kapitza, and Alessandro Sorniotti. Blockchain and trusted computing: Problems, pitfalls, and a solution for hyperledger fabric. *arXiv preprint arXiv:1805.08541*, 2018.
- [15] Steven M. Bellovin and Michael Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *Proceedings of the 1992 IEEE Symposium on Security and Privacy*, USA, 1992.
- [16] Fabrice Boudot, Pierrick Gaudry, Aurore Guillevic, Nadia Heninger, Emmanuel Thome, and Paul Zimmermann. Discrete logarithms in GF(p): 795 bits, 2019. [Online].
- [17] Nerla Jean-Louis, Yunqi Li, Yan Ji, Harjasleen Malvai, Thomas Yurek, Sylvain Bellemare, and Andrew Miller. SGXonerated: Finding (and partially fixing) privacy flaws in TEE-based smart contract platforms without breaking the TEE. *Cryptology ePrint Archive*, Paper 2023/378, 2023.
- [18] James Austgen, Andrés Fábrega, Mahimna Kelkar, Dani Vilardell, Sarah Allen, Kushal Babel, Jay Yu, and Ari Juels. Liquefaction: Privately liquefying blockchain assets, 2024.
- [19] Thomas Wu. The SRP authentication and key exchange system. Technical report, 2000.
- [20] SeongHan Shin and Kazukuni Kobara. Efficient augmented password-only authentication and key exchange for IKEv2. Technical report, 2012.
- [21] Victor Shoup. SPAKE2+, an augmented PAKE. Internet-Draft, IETF, 2020.
- [22] Daniel Bourdreux, Hugo Krawczyk, Kevin Lewi, and Christopher A. Wood. The OPAQUE augmented password-authenticated key exchange (aPAKE) protocol. Technical report, July 2025. Informational. Product of the Crypto Forum Research Group (CFRG).
- [23] Pawel Szalachowski. Password-authenticated decentralized identities. *arXiv preprint arXiv:2007.15881*, 2020. Preprint.
- [24] Manu Sporny, Dave Longley, Markus Sabadello, Drummond Reed, Orie Steele, and Christopher Allen. Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations. W3C recommendation, World Wide Web Consortium (W3C), July 2022. Accessed: 2026-03-12.
- [25] Manu Sporny, Dave Longley, and David Chadwick. Verifiable credentials data model v1.1. W3C recommendation, World Wide Web Consortium (W3C), March 2022. Accessed: 2026-03-12.
- [26] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindström, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. zklogin: Privacy-preserving blockchain authentication with existing credentials. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, page 3182–3196. ACM, December 2024.
- [27] Ivan Homoliak and Martin Perešini. Sok: Cryptocurrency wallets—a security review and classification based on authentication factors. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–8. IEEE, 2024.